SmartHire GenAI: Final Project Report

Name: S. Nachammai
Project type: Individual capstone, Generative AI
Live demo: https://smarthiregenai-o2aln6d6ehvvmpnazszzas.streamlit.app/
Code: GitHub repository `SmartHire_GenAI`

1. What I built

SmartHire GenAI is a career portal for students. You upload a resume and the app does four things:

1. Reads the resume and turns it into a clean, structured profile.
2. Finds jobs that match the profile, based on meaning and not just keywords.
3. Suggests how to improve the CV for a target role, and can rewrite the resume for that role.
4. Lets you ask career questions to an AI mentor that answers from my own career notes.

Everything is used through one Streamlit app, and a guardrails layer checks the input before it reaches the model.

2. How it is put together

The flow is simple. The resume is loaded and split into chunks. The LLM then converts it into a JSON profile, which I validate with Pydantic before using it. That profile is embedded and searched against a FAISS index of job descriptions. For CV suggestions, the resume and a job description go into a prompt from my prompt library. The mentor uses RAG: it retrieves the most relevant parts of my career notes and answers from those only.

The code is split into folders by job: `parsing`, `search`, `generate`, `mentor` and `safety` inside `src/`, and the Streamlit UI sits separately in `app/`.

3. Design choices and why

Gemini for the LLM and embeddings. I used `gemini-3.5-flash-lite` for generation and `gemini-embedding-001` for embeddings. It was easy to get an API key, and using one provider for both kept the setup simple.

FAISS for search. My dataset is small, so a local FAISS index is fast enough and needs no separate database. It also made deployment easy, because the index files are just part of the repository.

Structured output with validation. A resume is messy text. Asking the LLM for strict JSON and validating it means the rest of the app can trust the fields it receives.

RAG for the mentor. I wanted the mentor to stick to facts. By retrieving from my notes first, the answers stay grounded, and the mentor can say it does not know when the notes do not cover a question.

A pre-collected job dataset. Scraping LinkedIn or Naukri is against their terms, so I used a prepared dataset and embedded the descriptions.

Guardrails before every LLM call. The input check blocks unsafe requests (for example, asking it to add fake experience), prompt injection and off-topic questions. For resume rewrites, I also check that the rewritten text does not add skills or numbers that were not in the original.

4. Evaluation

I tested the project instead of only demoing it. The full reports are in the `reports/` folder.

Retrieval. For job search, the expected role appeared in the top three results for every test query (5 out of 5, 100%), and it was ranked first in the cases I checked. For the career notes, retrieval worked for 3 out of 5 questions (60%). It missed “What should a fresher learn first for Gen AI?” and “Should I do a Ph.D. after MCA?”.

Mentor answer quality. I used an LLM as a judge and scored answers from 1 to 5. The averages were relevance 4.25, groundedness 5.0 and actionability 3.5. Groundedness was perfect, which is what I cared about most for a RAG mentor. Actionability was the weakest. The lowest scores were for the question about explaining a DevOps internship in a Gen AI resume, because my notes do not cover that clearly.

Prompt comparison. I compared two versions of the CV suggestion prompt (v1 and v2) on 5 resumes, scoring each from 1 to 5. The second version improved groundedness from 4.4 to 5.0, which means the suggestions stuck to what was really in the resume. It was not better on everything though. Specificity dropped from 4.8 to 4.4 and actionability from 5.0 to 4.8, and structure stayed at 5.0 for both. I kept v2 because I cared more about suggestions that do not invent things than about extra detail.

Hallucination check. I tested this in two ways.

- *Resume rewrite.* I rewrote 5 resumes and checked whether the output added skills or numbers that were not in the original. Only one resume (the Gen AI sample) needed a second fix pass, and after the guard none of the five had any new invented items.
- *Questions outside the notes.* I asked the mentor 3 questions it should not be able to answer. It clearly said it could not help with 2 of them (the cricket world cup question and the “which company will definitely hire me” question). My check marked the salary question as a miss, but the answer begins by saying it does not quote exact salaries, so it did not make up a number. I think my check was a little too strict there, but I have not gone through the full answer, so I count the result as 2 out of 3.

5. What worked well

- The parser, job search and mentor all work together in the deployed app.
- The mentor’s answers stayed grounded in the notes, and the guardrails caught the unsafe and off-topic test inputs I tried.
- Splitting the code into `src/` and `app/` made it much easier to fix things later.

6. Problems I ran into

- **Repeated jobs in the results.** My job dataset is synthetic, so many rows share the same title, company and location but have different skills. At first the top results showed the same job again and again. I fixed this in the UI by showing only the best match for each title, company and location.
- **Match scores.** The raw similarity score (like 0.48) means nothing to a normal user. I removed it from the screen and show the jobs in ranked order instead.
- **Restructuring the code.** Moving everything into the final folder layout broke many imports, and I had to fix them one by one. Running my tests after each change helped me find what was broken.
- **Deployment.** The deployed app first failed because the API key was only on my computer. I had to add it to the app’s secrets.

7. Limitations

- The job data is synthetic and small, so the matches are only as good as that data.
- The career notes are limited, so the mentor cannot answer everything. Two of my five knowledge-base test questions were missed in retrieval.
- Mentor answers can be less actionable when the notes only give general advice.
- The mentor does not remember earlier questions in a session.
- Guardrails reduce misuse but are not a complete safety system.
- The free Gemini API has usage limits, so heavy use can be slow or fail.

8. What I would do next

- Add conversation memory to the mentor.
- Grow the career notes and the job dataset with real postings.
- Improve retrieval for the questions that missed.
- Try comparing two embedding models and report which gives better matches.

9. What I learned

This was my first time building a full GenAI project from start to finish. The biggest lesson was that the model is only one part. Most of the work was in the things around it: validating the output, retrieving the right context, protecting the input, and testing the results.